

3C7 Digital Systems Design – Lab F

Harry Leddy – 23372611

Table of Contents:

1.0: Part A:	2
1.1: Original DFF:.....	2
1.2: Negative-Edge Triggered DFF:	2
1.3: Synchronous Reset:	3
2.0: Part B:	4
2.1: Design Description:.....	4
2.2: Implementation:	4
2.3: Demo:	5
3.0: Part C:.....	6
3.1: LFSR Development & Testing:	6
3.1: Advanced Section:	8
4.0: Part D:.....	10
4.1: Basys 3 Board Integration:	10
4.2: Demo:	11

1.0: Part A:

In the first part of this laboratory, the behaviour of a D-type flip-flop was examined. Three test signals were created (in accordance with the brief): a periodic “clock” signal, a “reset” signal and an input “d” signal which controls the output of the flip-flop. The timing of these signals were based on the timing diagram given in the lab briefing.

1.1: Original DFF:

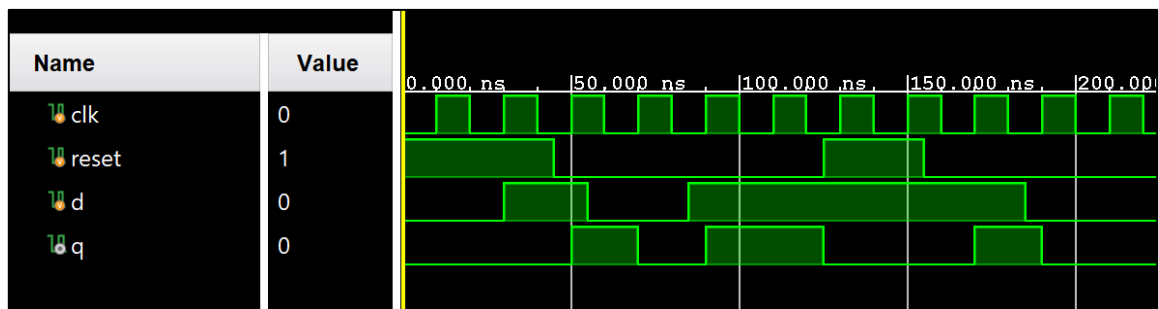


Figure 1 - Original DFF Testbench

The waveform in Figure 1 shows the behaviour of a positive-edge triggered D-type flip-flop with asynchronous reset. The output “q” is updated only on the rising edge of the “clock” signals, it typically takes the value of the input “d”. When the “reset” signal is high, the output is forced to zero, regardless of the value of “d”. Since this reset is asynchronous, “q” is forced to zero immediately on the rising edge of the reset signal, it does not need to wait on the clock update.

Once reset returns to low, “q” again takes on the value of “d”, but is only updated at the rising edges of the clock signal. Between clock edges, changes in “d” do not affect the output, this shows that the flip-flop stores its state until the next clock event.

1.2: Negative-Edge Triggered DFF:

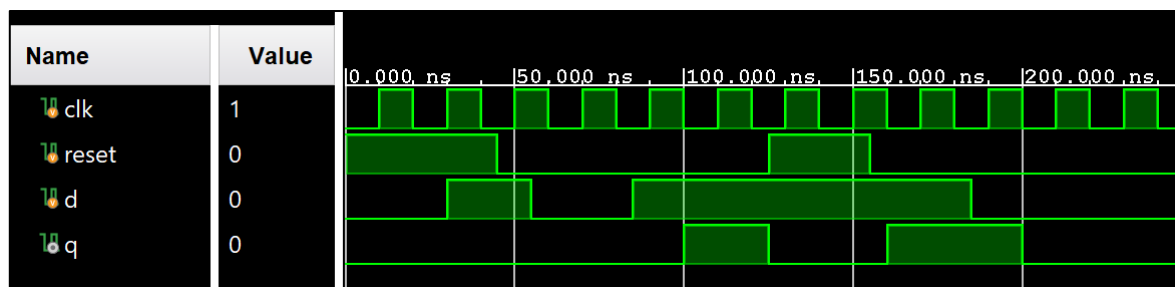


Figure 2 - Negative-Edge Triggered DFF Testbench

The waveform in Figure 2 shows the behaviour of a negative-edge triggered D-type flip-flop, also with an asynchronous reset. In this case, the output signal “q” is updated to the value of “d” on the falling edge of the clock signal, rather than the rising edge, as before. As a result, the output is updated based on the input “d” at different points compared to the positive-edge case. This leads to differences in the output waveform, as changes in “d” are only captured at falling clock edges.

For the given input signals used in this testbench, the initial positive value for the “q” signal seen in Figure 1 is not present in Figure 2. This is because the same high value of “d” with a low value

of “reset” does not coincide with a falling clock edge. As a result, “q” only first transitions high at a later point in time, when “d” is sampled on a falling edge. The reset behaviour in this example remains asynchronous, so setting “reset” to high still forces “q” to low immediately.

1.3: Synchronous Reset:

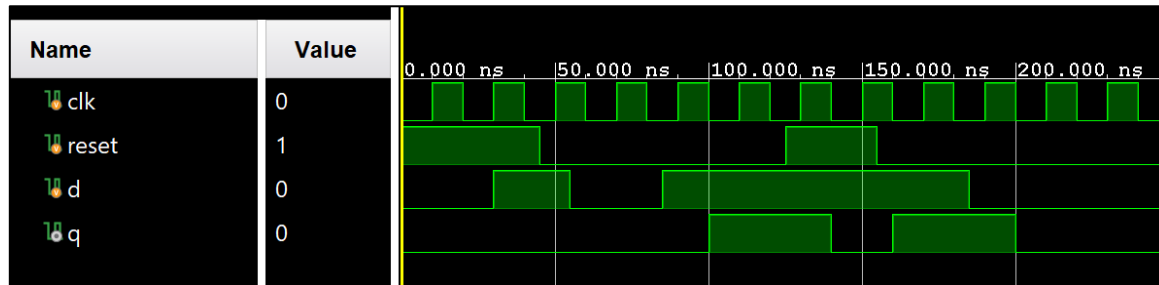


Figure 3 – Synchronous-Reset & Negative-Edge Triggered DFF Testbench

The waveform seen in Figure 3 shows the behaviour of the negative-edge triggered D-type flip-flop after modifying the reset to be synchronous. In this case, the output “q” is still updated on falling clock edges, but the reset signal no longer affects the output immediately.

Instead, when “reset” is set to high, “q” is forced low only at the next falling edge of the clock. This creates a clear timing difference compared to the asynchronous reset cases shown in Figure 1 and Figure 2 where “q” changed as soon as reset was activated. This demonstrates that a synchronous reset is dependent on the clock signal, whereas an asynchronous reset can override the stored state immediately upon activation.

Comparing Figures 1-3, the asynchronous reset forces the “q” output low immediately when reset is activated, whereas the synchronous reset only affects “q” on the next falling clock edge (for a negative-edge system).

2.0: Part B:

2.1: Design Description:

The design was implemented for an 8-bit counter system as outlined in the lab briefing. The relevant modules and wires were all instantiated and connected. There were 8 D-type flip-flops used to store the current binary state of the system. Using 8-bit representation like this meant 256 values could be represented (0 – 255 °F). Flip flops are useful for binary representations as increasing a bit will propagate the increase if necessary, matching binary counting behaviour.

Debouncer modules were also used to avoid distortion of the push-button inputs due to their spring mechanism. A 7-segment display module was also used to display the stored value in human-readable decimal format. Finally, combinational logic was also used to implement the required behaviour. This involved using the UP/RIGHT buttons to increase the current stored value and the DOWN/LEFT buttons to decrease it. The centre button was also used to set the temperature to a fixed value of 22 °F.

2.2: Implementation:

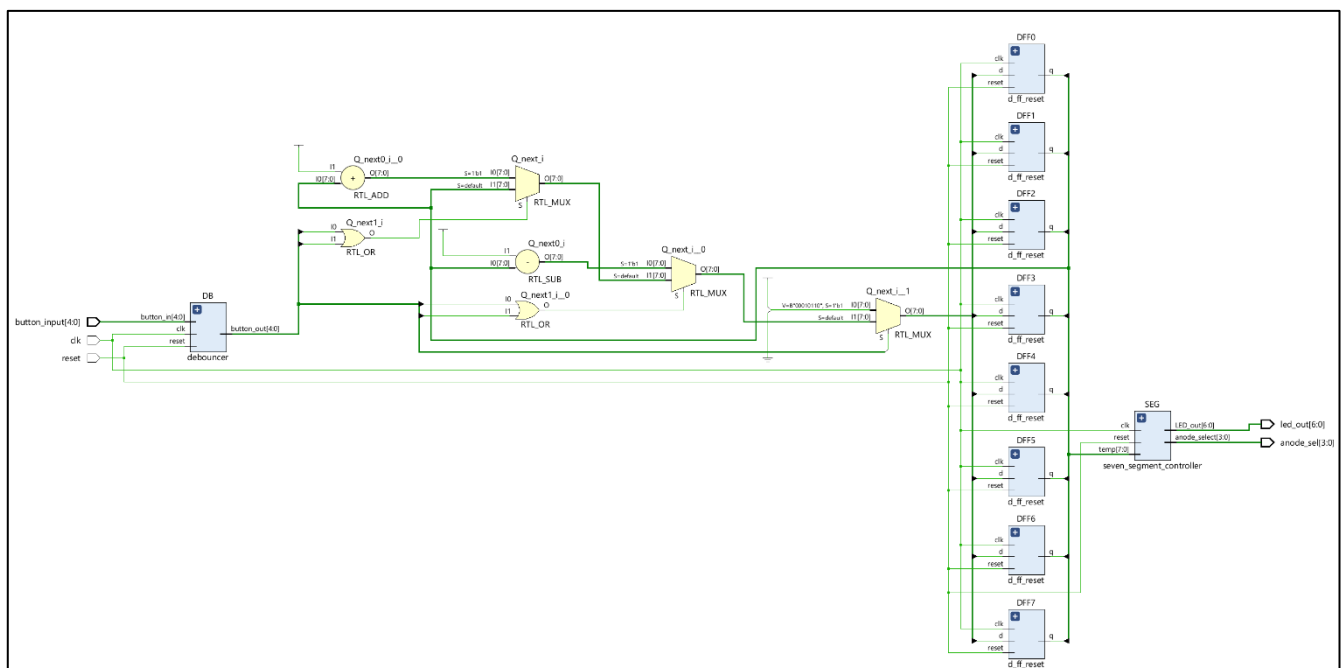


Figure 4 - RTL Schematic of Counter Design

The outlined design was then developed, synthesised and implemented in Vivado. The RTL schematic is shown in Figure 4. While it is more detailed than the simplified block diagram provided in the lab briefing, it reflects the same overall structure, with the combinational logic feeding into the D flip-flops and the stored value being passed to the seven-segment display controller.

The given constraints file was also used to map the physical switches and LEDs on the Basys 3 board to the inputs and outputs of this counter design.

2.3: Demo:

The implemented system behaved as expected when deployed on the Basys-3 board. The counter successfully incremented when the UP or RIGHT buttons were pressed and decremented when the DOWN or LEFT buttons were pressed. The CENTRE button correctly set the temperature to a fixed value of 22 °F.

The functionality is demonstrated in the following figures. Figure 5 shows the system set to the fixed value of 22, Figure 6 shows the lower bound of 0, and Figure 7 shows the upper bound of 255, confirming the full functional range of the 8-bit counter.

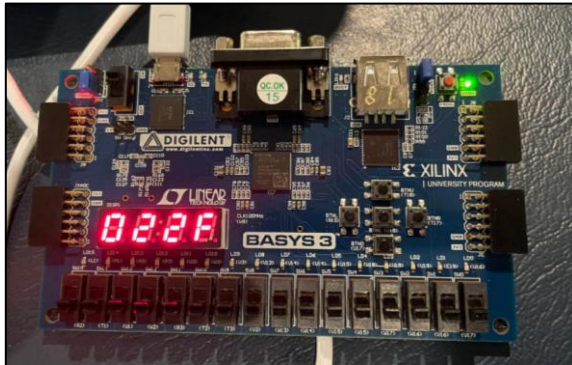


Figure 5 - Counter System - 22 F

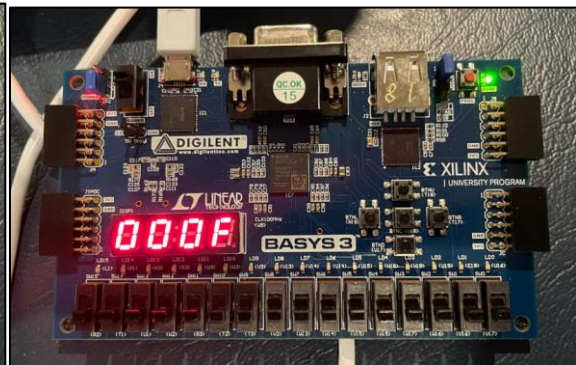


Figure 6 - Counter System - 0 F

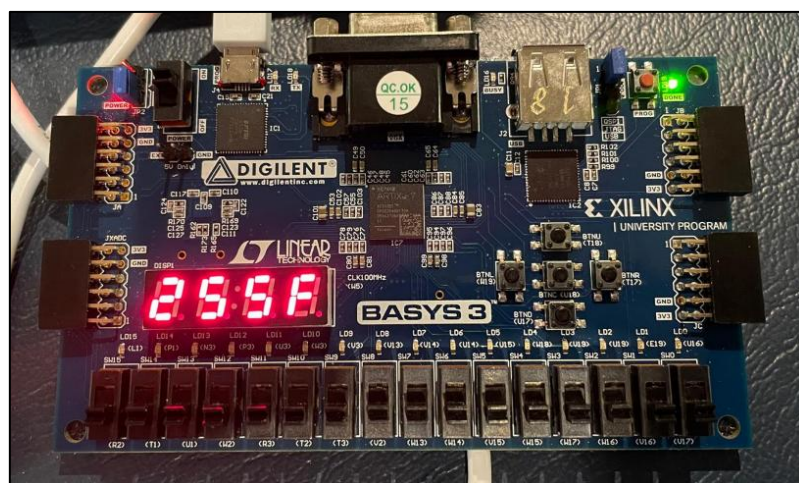


Figure 7 - Counter System - 255 F

The current value was displayed correctly on the seven-segment display, including the unit indicator (F). The reset function, which was mapped to switch “S0”, also worked as intended. It returned the temperature value to zero when activated and then deactivated. Overall, the implementation behaved as expected.

3.0: Part C:

3.1: LFSR Development & Testing:

For this design, an LFSR length of $N = 23$ was assigned, with XOR feedback. The seed value was calculated by taking the bitwise XNOR of the board number (18) and the last three digits of the student ID (611), giving a hexadecimal seed of D8E. The calculations for this can be seen in Figure 8.

0000 0001 0010	- 18 in binary = 12 hex (board number)
0000 0110 0011	- 611 in binary = 63 hex (last 3 digits of student num)

0010 0111 0001	- XOR value = 271 hex
1101 1000 1110	- NXOR value = D8E hex

Figure 8 - LFSR Seed Calculations

A linear feedback shift register (LFSR) is a shift register where the new input bit is generated from specific bits in the current state using feedback logic. In this design, the register shifts left on each clock cycle, while the least significant bit is replaced by the feedback value. This system produces a pseudo-random sequence of values.

According to the tap table in “*xapp052.pdf*”, a 23-bit LFSR uses taps at positions 23 and 18 to compute the feedback value. Using the zero-based indexing in Verilog, these correspond to $Q[22]$ and $Q[17]$. The feedback bit is therefore given by:

$$Q_{ns} = Q[17] \oplus Q[22]$$

The block diagram for this 23-bit LFSR system can be seen in Figure 9. As previously described, all bits are shifted and the new bit is calculated by XORing the two tap positions.

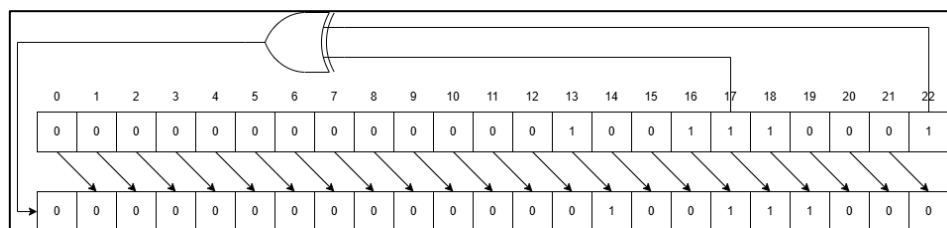


Figure 9 - 23-bit LFSR Block Diagram

This system was then created following the guidance given in Lecture 5. The given template was modified to have a length of 23-bits, to be asynchronous and to have an active reset. The maximum-tick-register was also added. This compared the current state to the beginning seed state. This would only occur when the LFSR had completed a full cycle, which is $2^n - 1$ states.

In order to create a testbench for this system, a simple clock was created which toggled every one nanosecond, giving a full clock period of two nanoseconds. Since the 23-bit LFSR cycles through $2^{23} - 1 = 8,388,607$ states, the total sequence duration was given by:

$$2^{23} - 1 = 8,388,607 \text{ cycles} = 16,777,214 \text{ ns}$$

Then the initialising reset signal also needed be taken into account. This lasted for 10 clock cycles, two nanoseconds each. Therefore, the total testbench duration was calculated as:

$$16,777,214 + 20 = 16,777,234 \text{ ns}$$

This calculated value was how long the testbench had to be simulated for in order to trigger the maximum-tick-register. This can be seen in Figure 10, where the “*max_tick_reset*” can be seen to be high in the cycle between 16,777,233 ns and 16,777,235 ns. Immediately after this, the state restarts from the seed and continues through the same pseudo-random cycle.

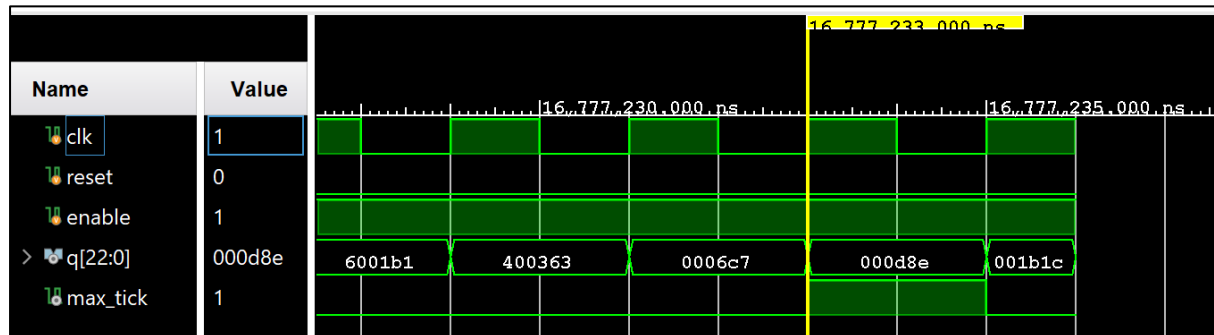


Figure 10 - LFSR Testbench - Max Tick Reset

The previously mentioned initialising reset signal can be seen in Figure 11. The full vector representation of the system's state can be seen in this figure. The seed value is held as the current state while reset is high. Once the reset signal is disabled, the state begins to shift with each clock cycle and eventually introduces new feedback bits, as expected.

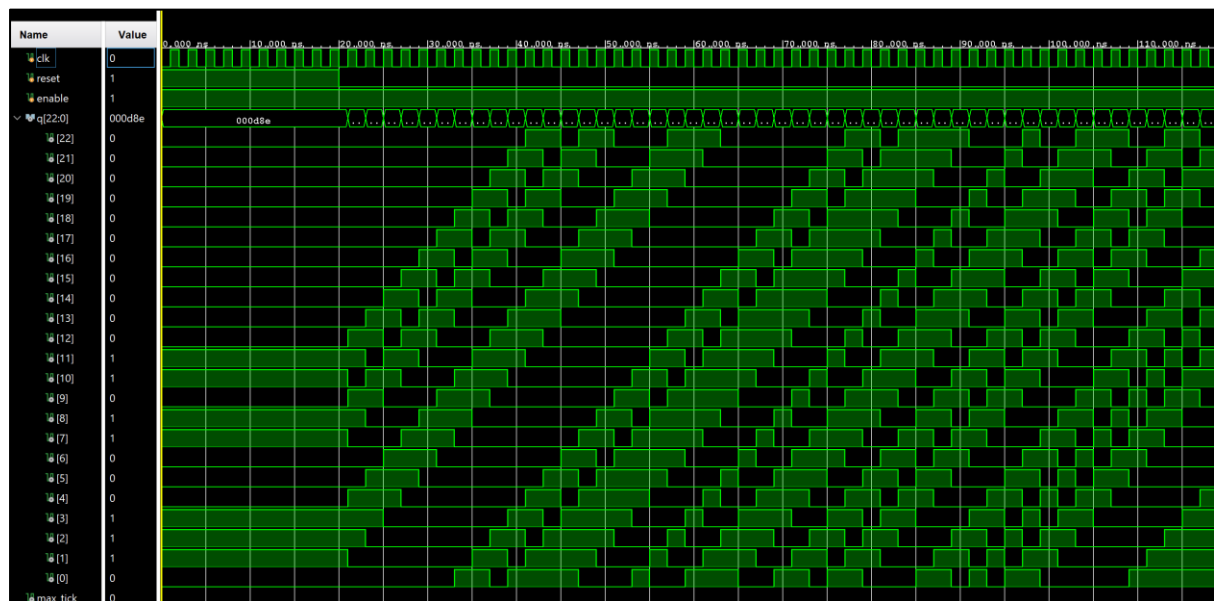


Figure 11 - LFSR Testbench - Reset Signal

The seed was then changed to the forbidden value of all zeros. For a XOR-feedback LFSR, this state is invalid as the feedback bit remains zero forever, so the state cannot leave the all-zero seed. As shown in Figure 12, the output remains constant at zero for the entire simulation.

In this case, the maximum-tick-register also remains high continuously, as the current state is always equal to the seed state. This confirms the expected locked-state behaviour of the XOR LFSR when it is initialised with the forbidden seed.

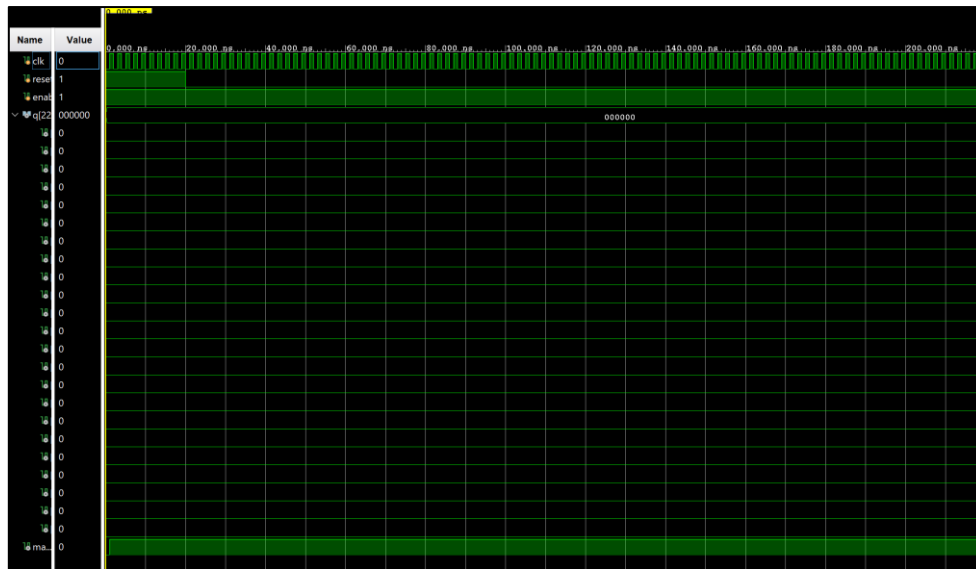


Figure 12 - LFSR Testbench - Forbidden Seed

3.1: Advanced Section:

This section required the development of a “*bit counter*” module. This was an asynchronous, positive-reset system, matching the LFSR it was counting. If the MSB of the LFSR’s state was a one, the module would increment the “*one counter*” and the corresponding behaviour would also occur if the MSB was a zero. This would be reset upon the maximum-tick-register being activated. The schematic for the system testbench can be seen in Figure 13.

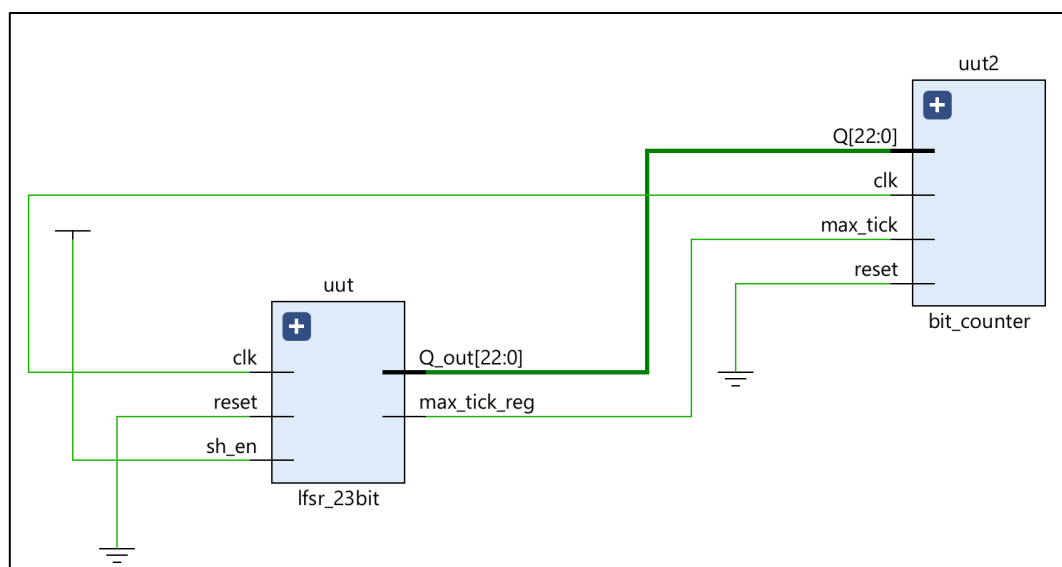


Figure 13 - LFSR with Bit Counter Schematic

To count the number of ones and zeros seen as the MSB over the full LFSR cycle, counters were required. Since the maximum count is 8,388,607, the required counter length is:

$$8,388,607 \text{ cycles} \Rightarrow \log_2(8,388,607) \approx 23$$

Analysis of the waveform showed that the numbers of ones and zeros at the MSB differed by only one over the full LFSR period (4,194,304 and 4,194,303 respectively). This is the expected behaviour, since such sequences are approximately balanced between high and low values. This supports the use of the LFSR as a pseudo-random source. The “probability” of a one or zero occurring is approximately equal.

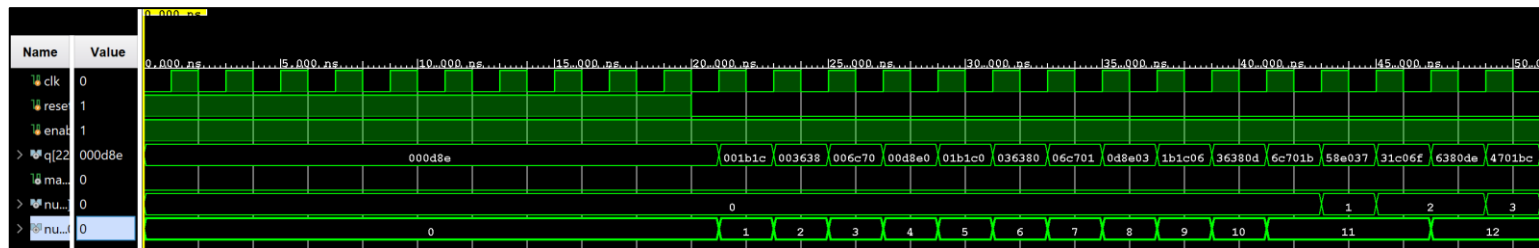


Figure 14 - LFSR Testbench with Bit-Counter

The counter module begins counting only after the reset signal was disabled, as shown in Figure 14. The two counters are also reset when max-tick-register becomes HIGH, this causes the counting process to restart at the beginning of the next LFSR cycle, as shown in Figure 15.

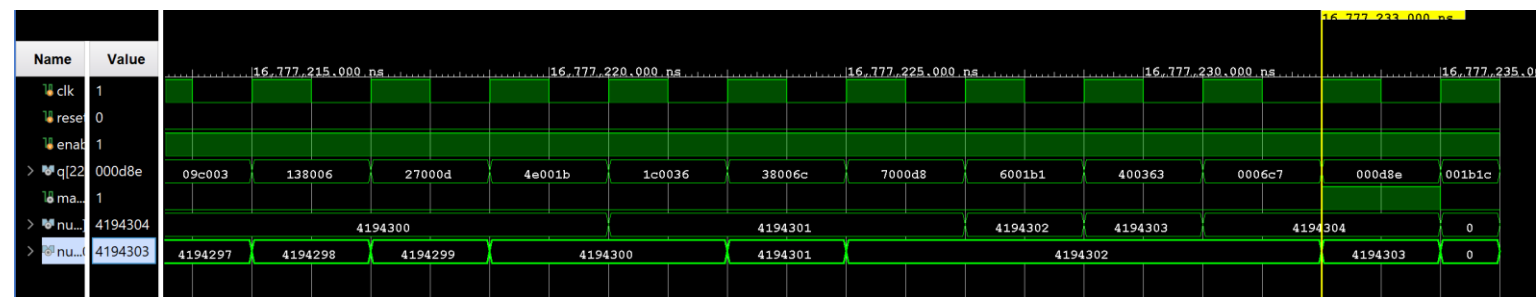


Figure 15 - LFSR Testbench with Bit-Counter - Maximum Tick Reset

4.0: Part D:

4.1: Basys 3 Board Integration:

To put the design onto the Basys 3 board, a top-level module was created containing both a clock divider and the 23-bit LFSR. The board's onboard clock input was scaled down to 1Hz, using a clock scaling module, so that individual state changes could be observed directly on the LEDs.

The clock-scaling module consisted of an internal clock register. This would increment with each cycle of the on-board oscillator clock. Once this register value reached a threshold (defined by a module input), this would flip the state of the module's scaled output clock.

Since the LFSR output is 23 bits long and the board only contained 16 LEDs, a multiplexer module was created to select which portion of the LFSR state would be displayed. The module simply checked the state of the "select" input and either outputted the first 15 values of the LFSR state or the remaining 8. This allowed the full state to be inspected in two parts using a switch input.

The LFSR, multiplexer and clock-scaling module were all integrated using a top level orchestrator module. The schematic for this system can be seen in

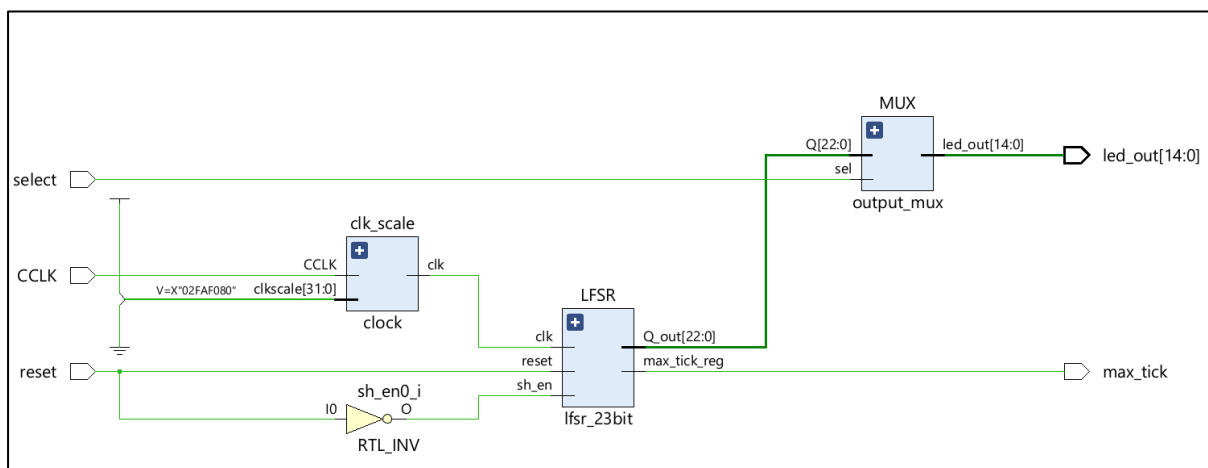


Figure 16 - Top Level Module - LFSR Integration

Using Vivado's I/O editor, a XDC constraints file was generated. This mapped the clock, reset switch, select switch, state LEDs and the maximum-tick-register LED to the physical pins on the Basys 3 board. The configuration can be seen outlined in Figure 17.

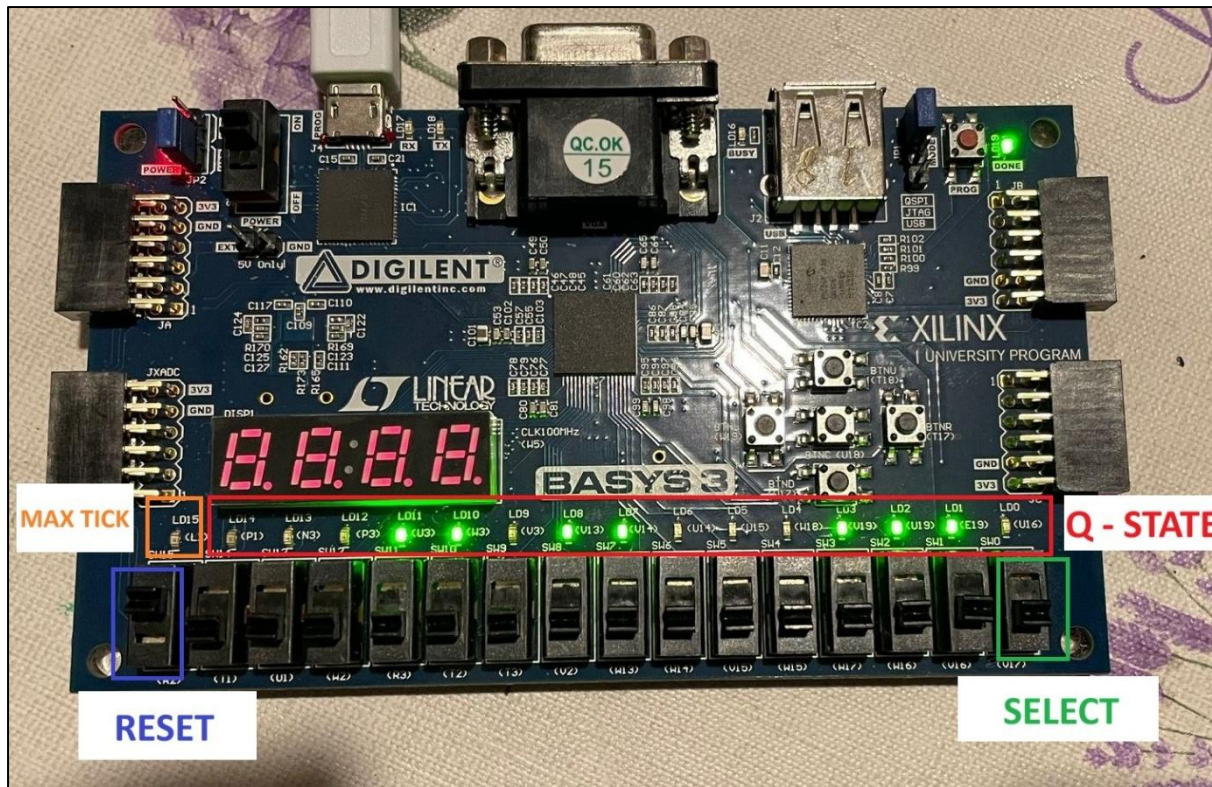


Figure 17 - LFSR - Basys 3 Board Configuration

The maximum-tick-register LED was not verified on the physical hardware. Since the clock was scaled to have a cycle frequency of 1 Hz and the LFSR has to complete 8,388,607 cycles before looping, it would take nearly 14 weeks of constant runtime before the LED would be triggered.

4.2: Demo:

The reset functionality was first tested on the physical Basys 3 board. This can be seen in Figure 18. When the left-most switch is HIGH, the state of the LFSR is forced to the seed-value (D8E). This confirms the expected behaviour as the seed should act as the initial state value.

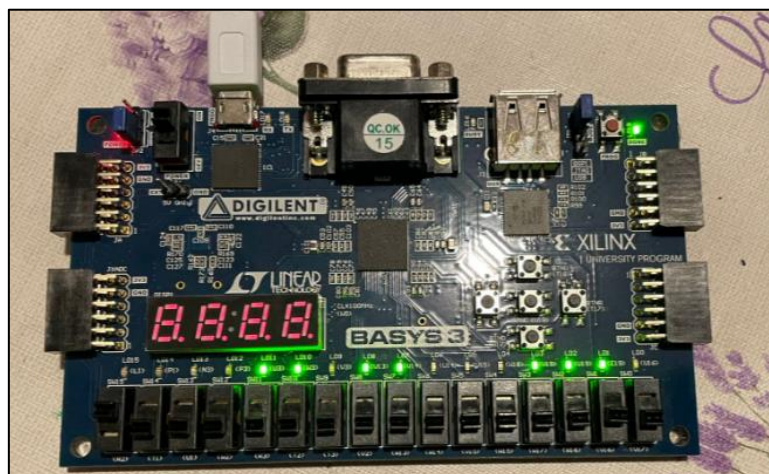


Figure 18 - Reset Functionality Demo

The reset switch was then set to LOW, this can be seen in Figure 19. The state of the LFSR could then be seen to update every second. This involved the bits shifting to the left, with the right most-bit being calculated based on the feedback bit for each cycle. In this configuration, only 15 of the 23 state bits could be shown on the board.

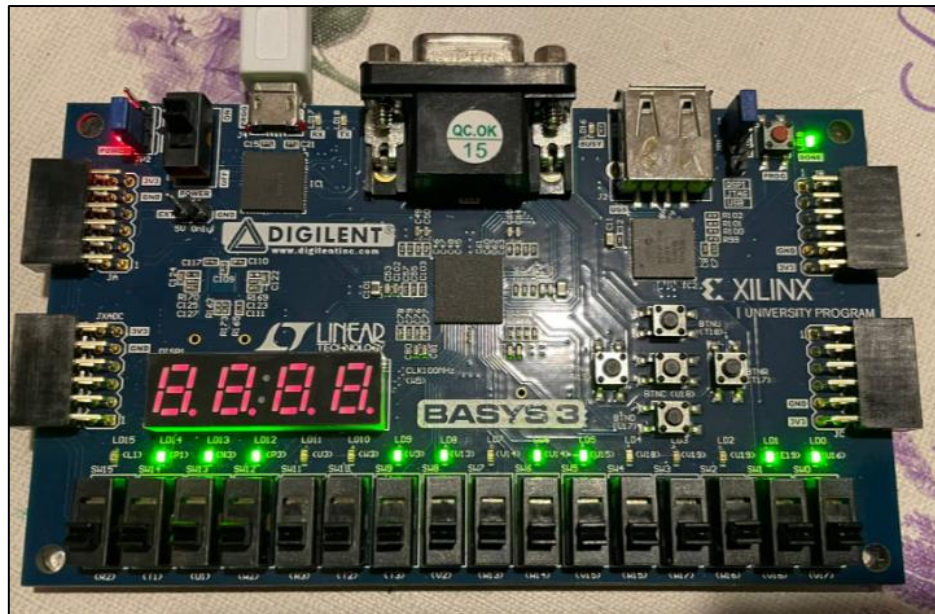


Figure 19 – LFSR Functionality Demo

For the remaining 8 bits to be viewed, the rightmost switch would have to be set to HIGH. This acted as the “select” input to the multiplexer. As seen in Figure 20, the remaining bits could be seen with the “select” input activated.

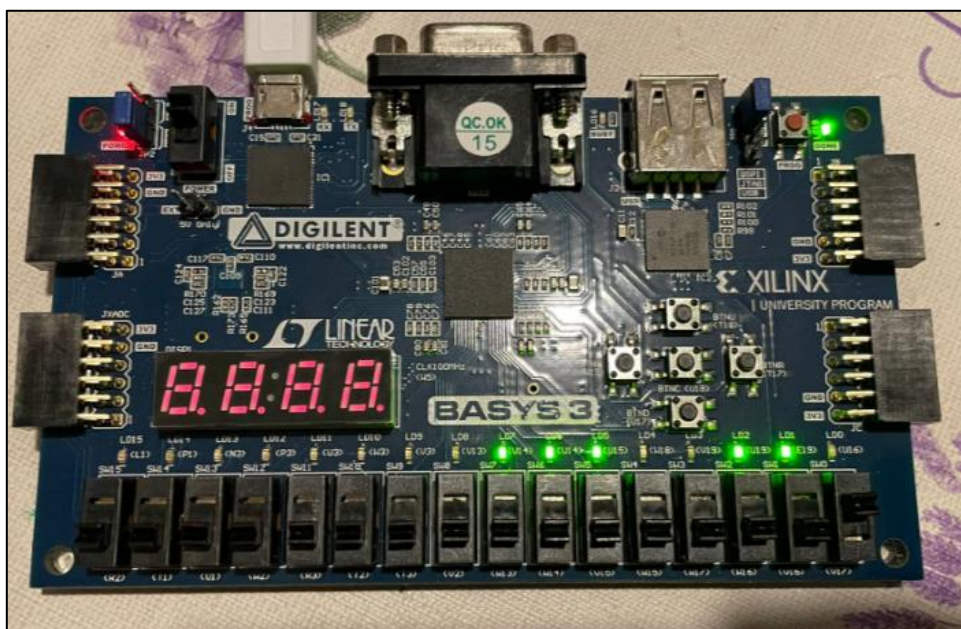


Figure 20 - MUX Functionality Demo

The 16th LED is reserved for the maximum-tick-register which, as previously explained, was not included in the physical demonstration.